Module 18 - Assignment

S M Wasir Jayed Rafi
ID: 01410369963
Full Stack Web Development with Python, Django, React & AI
Submission Date: 02/08/26

1. Explain the following Django model relationships with suitable real-life examples: One-to-One, One-to-Many, Many-to-Many

Ans: Django gives us three field types to connect models, and each one matches a different kind of real-life relationship.

One-to-One means one record on each side and nothing more. A real-life example is a person and their NID card, one person has exactly one NID and one NID belongs to one person. In Django it's commonly used for a user and their profile, where you keep extra details like phone number separate from the main user model. We use `OneToOneField`.

One-to-Many means one record on one side can connect to many on the other side. A real-life example is a blog post and its comments, one post can have many comments but each comment belongs to only one post. We use `ForeignKey`, and it always goes on the "many" side.

Many-to-Many means both sides can connect to many records. A real-life example is students and courses, one student can enroll in many courses and one course can have many students. We use `ManyToManyField`, and Django creates a separate join table in the background on its own.

```
from django.db import models
from django.contrib.auth.models import User

class Profile(models.Model):                                    # One-to-One
    user = models.OneToOneField(User, on_delete=models.CASCADE)
    phone = models.CharField(max_length=20)

class Post(models.Model):
    title = models.CharField(max_length=200)
    author = models.ForeignKey(User, on_delete=models.CASCADE)  # One-to-Many
    created_at = models.DateTimeField(auto_now_add=True)

class Comment(models.Model):
    post = models.ForeignKey(Post, on_delete=models.CASCADE)    # One-to-Many
    body = models.TextField()

class Course(models.Model):
    name = models.CharField(max_length=100)
```

```
class Student(models.Model):                                    # Many-to-Many
    name = models.CharField(max_length=100)
    courses = models.ManyToManyField(Course)
```

The `on_delete=models.CASCADE` part tells Django to delete the comments too when the post is deleted, so we don't end up with comments pointing at something that no longer exists.

2. What are select_related() and prefetch_related()? Explain their differences and mention when each should be used.

Ans: Both are used to reduce the number of database queries when accessing related objects. Without them Django hits the database again every time you touch a related field inside a loop, which is called the N+1 problem and it slows the page down a lot.

`select_related()` uses a single SQL JOIN, so the related data comes back together with the main query. `prefetch_related()` runs a separate query instead and matches everything up in Python, which is why it can handle relationships that return many rows.

```
# without optimization - one extra query per comment (N+1 problem)
for c in Comment.objects.all():
    print(c.post.title)              # hits the database every loop

comments = Comment.objects.select_related('post')       # one query, JOIN
students = Student.objects.prefetch_related('courses')  # two queries
```

Point	**select_related()**	**prefetch_related()**
How it works	Uses a SQL JOIN	Separate query, matched in Python
Number of queries	Just 1	2 or more
Works with	ForeignKey and OneToOneField	ManyToManyField and reverse ForeignKey
When to use	Each object has one related object	Each object has many related objects

So the simple rule is, if the relationship gives back one object use `select_related()`, and if it gives back a list use `prefetch_related()`.

3. What is Model Inheritance in Django? Explain the different types of model inheritance and mention one use case for each.

Ans: Model inheritance is when one model inherits fields from another, just like normal Python class inheritance. It's used so you don't repeat the same fields in different models. Django supports three types.

1. Abstract Base Classes — you set `abstract = True` in Meta, and Django doesn't create a table for the parent at all, it just copies the fields into the children. Use case: when many models need the same `created_at` and `updated_at` fields, you write them once and inherit everywhere.
2. Multi-table Inheritance — the parent is a normal model with its own table and the child gets its own table too, linked automatically. Use case: a `Place` model with name and address, and a `Restaurant` that inherits it and adds its own fields.
3. Proxy Models — you set `proxy = True`, so no new table is created and it uses the parent's table, but you can change the Python behaviour like default ordering. Use case: showing the same data in the admin panel a second time with a different ordering.

```
class TimeStamped(models.Model):              # 1. abstract base class
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        abstract = True                       # no table for this one

class Product(TimeStamped):                   # gets created_at from the parent
    name = models.CharField(max_length=100)
    price = models.IntegerField()

class Place(models.Model):                    # 2. multi-table inheritance
    name = models.CharField(max_length=100)
    address = models.CharField(max_length=200)

class Restaurant(Place):
    serves_pizza = models.BooleanField(default=False)

class OrderedProduct(Product):                # 3. proxy model
    class Meta:
        proxy = True
        ordering = ['price']
```

Out of the three, abstract base classes are the ones used most often in real projects.

4. What are Aggregations and Annotations in Django ORM? Explain their purposes and provide examples of common aggregation functions.

Ans: Both are used to calculate values inside the database instead of looping through records in Python and counting manually. The difference is what they give back.

Aggregation uses `aggregate()` and it collapses the whole queryset into one summary value. It returns a dictionary, so you can't chain anything after it. You use it when you want one overall answer, like the average price of all products.

Annotation uses `annotate()` and it adds a calculated value to every object in the queryset. It returns a queryset, so you can still filter or order by that new value afterwards. You use it when you want a per-row calculation, like how many comments each post has.

The common aggregation functions are `Count()`, `Sum()`, `Avg()`, `Max()` and `Min()`, imported from `django.db.models`.

```
from django.db.models import Count, Sum, Avg, Max

Product.objects.aggregate(Avg('price'))       # {'price__avg': 450.0}
Product.objects.aggregate(total=Sum('price'), highest=Max('price'))

posts = Post.objects.annotate(comment_count=Count('comment'))
for p in posts:
    print(p.title, p.comment_count)
```

The benefit is that the database does the calculation, which is much faster than pulling thousands of rows into Python first.

5. Explain how Ordering and Slicing work in Django QuerySets. Why are they useful in web applications?

Ans: Ordering is done with `order_by()`. You pass the field name to sort ascending and put a minus sign in front for descending, and you can pass multiple fields so the second one breaks ties. If a model should always come back in a certain order you can set `ordering` in its Meta class instead of writing it every time.

Slicing uses normal Python list slicing, and Django converts it into SQL `LIMIT` and `OFFSET`. This works because querysets are lazy, meaning the query doesn't actually run until you use the data, so the slice gets added into the SQL before it runs.

```
Post.objects.order_by('-created_at')             # newest first
Product.objects.order_by('name', '-price')       # two fields

Post.objects.order_by('-created_at')[:5]         # latest 5 posts
Post.objects.all()[10:20]                        # rows 11 to 20, like page 2
```

They're useful because almost every page needs data in a specific order and in limited amounts, like latest posts on a homepage, top selling products, or pagination showing 10 items per page. Slicing also keeps things fast, since the database only sends back the rows you asked for instead of loading the whole table into memory. One thing to remember is you can't filter a queryset after slicing it, so filtering has to come first.

6. What are Related Objects and Lookup Expressions in Django ORM? Explain how they help retrieve related data efficiently.

Ans: Related objects means once a relationship is defined, Django gives you a way to move between the two models without writing any JOIN yourself. Going from the child to the parent is forward access and you just use the field name like `comment.post`. Going from the parent to the children is reverse access, and Django names it `comment_set` by default, but if you set `related_name` you can call it something nicer like `post.comments`.

Lookup expressions are the double underscore `__` conditions used inside `filter()`. They let you do more than exact matching, and they also let you follow a relationship into another model's fields.

```
class Comment(models.Model):
    post = models.ForeignKey(Post, on_delete=models.CASCADE,
                             related_name='comments')

comment.post.title                # forward access
post.comments.all()               # reverse access

Post.objects.filter(title__icontains='django')      # case-insensitive contains
Product.objects.filter(price__gte=500)              # greater than or equal
Post.objects.filter(created_at__year=2026)          # date lookup

Comment.objects.filter(post__author__username='wasir')   # across relationships
```

Some common lookups are `exact`, `icontains`, `gt`, `gte`, `lt`, `lte`, `in` and `isnull`. They help with efficiency because the filtering and joining happens inside the database instead of in Python, and that last example crosses two relationships in a single query.

7. What are Raw SQL Queries in Django? Compare Django ORM and Raw SQL by discussing their advantages and disadvantages.

Ans: Raw SQL means writing the SQL yourself instead of using the ORM methods, and Django allows it for cases the ORM can't handle. `Model.objects.raw()` runs a SELECT and gives back real model objects, and `connection.cursor()` lets you run any SQL directly.

```
from django.db import connection

posts = Post.objects.raw(
    "SELECT * FROM blog_post WHERE title LIKE %s", ['%django%'])

with connection.cursor() as cursor:
    cursor.execute(
        "SELECT author_id, COUNT(*) FROM blog_post GROUP BY author_id")
    rows = cursor.fetchall()
```

You should always pass the values as parameters like above, and never join them into the query string with the user's input, because that opens the door to SQL injection.

Point	Django ORM	Raw SQL
How you write it	Python methods, no SQL needed	Actual SQL as a string
Readability	Shorter and cleaner	Gets hard to read as it grows
Portability	Same code works on SQLite, PostgreSQL, MySQL	Often tied to one database
Security	Escapes values automatically	You have to parameterize it yourself
Performance	Slight overhead, and it can fire extra queries	Can be faster for very complex queries
Best used for	Everyday CRUD work	Complex reports and heavy joins

So the ORM should be the default because it's safer and shorter, and raw SQL is only for the rare queries that are too complicated for it.

8. Explain the purpose of Django's built-in User model. Describe the role of User Registration, Login, Logout, Password Change, and Password Reset in a web application.

Ans: Django's built-in User model comes from `django.contrib.auth.models` and its purpose is to give you a ready-made user system so you don't have to build authentication from scratch. It already has fields like `username`, `email`, `password`, `is_active`, `is_staff`, `is_superuser` and `date_joined`, and it connects straight into the admin panel and the permissions system. The important part is that the password is hashed before saving, so it's never stored as plain text.

Then the five main things a web app needs around it:

User Registration is creating a new account, and Django's `UserCreationForm` handles the username and password fields and hashes the password before saving.

Login verifies who someone is. `authenticate()` checks the username and password and returns the user if it matches, then `login()` starts the session so they stay logged in across pages.

Logout ends that session with `logout()`, which clears the session and makes `request.user` anonymous again. Without it a user would stay logged in on a shared device.

Password Change is for someone already logged in who just wants a new password, and it asks for the old password first to confirm it's really them.

Password Reset is for someone who forgot their password and can't log in at all. Django emails a one-time token link that lets them set a new password without the old one, and the token expires so the same email can't be reused later.

```
from django.contrib.auth import authenticate, login, logout
from django.shortcuts import render, redirect

def login_view(request):
    if request.method == 'POST':
        user = authenticate(request,
                            username=request.POST['username'],
                            password=request.POST['password'])
        if user is not None:
            login(request, user)          # starts the session
            return redirect('home')
    return render(request, 'login.html')

def logout_view(request):
    logout(request)                       # clears the session
    return redirect('login')
```

9. What is LoginRequiredMixin? How does Session-Based Authentication work in Django? Why is it considered secure?

Ans: `LoginRequiredMixin` is used with class-based views to make sure only logged-in users can open a page. If someone isn't logged in, Django redirects them to the login URL instead of showing it. It's basically the class-based version of the `@login_required` decorator used on function-based views, and it has to be written first in the class definition or the check won't run properly.

```
from django.contrib.auth.mixins import LoginRequiredMixin
from django.views.generic import ListView

class DashboardView(LoginRequiredMixin, ListView):
    model = Post
    login_url = '/login/'
```

Session-based authentication is how Django keeps a user logged in. HTTP is stateless so the server doesn't remember anything between requests, and sessions are what fill that gap. The flow works like this:

1. The user submits the login form and `authenticate()` checks the password against the hashed one in the database.
2. If it matches, `login()` creates a session and saves it on the server, in the `django_session` table by default.
3. Only the random session ID is sent back to the browser, in a cookie called `sessionid`.
4. On every request after that the browser sends the cookie, `SessionMiddleware` loads the session from it, and `AuthenticationMiddleware` uses that to set `request.user`.
5. When `logout()` runs the session is deleted on the server and the cookie becomes useless.

It's considered secure because the actual data stays on the server and the cookie only holds a random ID, so there's nothing sensitive sitting in the browser. The cookie can also be set as `HttpOnly` so JavaScript can't read it and `Secure` so it only travels over HTTPS. On top of that sessions expire, logging out invalidates them immediately on the server side, and Django's CSRF protection stops other sites from making requests using your session.

10. Define Cookies and Middleware in Django. Explain how they work together during a request-response cycle and mention two practical uses of Middleware.

Ans: Cookies are small pieces of data the server tells the browser to store, and the browser sends them back with every request to that same site. Since HTTP doesn't remember anything on its own, cookies are how a website recognizes you between page loads. In Django you read them with `request.COOKIES` and set them with `response.set_cookie()`.

```
def set_theme(request):
    response = redirect('home')
    response.set_cookie('theme', 'dark', max_age=3600)   # expires in 1 hour
    return response

def home(request):
    theme = request.COOKIES.get('theme', 'light')        # 'light' if not set
    return render(request, 'home.html', {'theme': theme})
```

Middleware is a layer of code that sits between the request and the view. Every request passes through each middleware before reaching the view, and every response passes back out through them. They're listed in the `MIDDLEWARE` setting, and the order matters because they run top to bottom going in and bottom to top coming out.

Here's how they work together in the request-response cycle:

1. The browser sends a request and automatically includes the `sessionid` cookie.
2. `SessionMiddleware` reads that cookie and loads the matching session, attaching it as `request.session`.
3. `AuthenticationMiddleware` uses the session to figure out who the user is and sets `request.user`.
4. The view runs and can just use `request.user` without doing any of that work itself.
5. The response goes back out through the middleware, session changes get saved, and a `Set-Cookie` header is added if a new cookie is needed.

So cookies are what the browser holds onto, and middleware is what reads and updates them on the server side before the view ever runs.

Two practical uses of middleware:

1. Authentication and session handling — instead of every view checking cookies and looking up the user, middleware does it once per request and hands the view a ready `request.user`.
2. Security checks — `CsrfViewMiddleware` verifies the CSRF token on every POST request, which stops other websites from submitting forms using your logged-in session.